

Camera AI: Built System, Setup & AWS Deployment Guide

A complete local-first implementation for face presence, licence-plate detection and OCR, vehicle attributes, repeat matching, Camera Trust Score, adaptive processing modes, evidence JSON and staged AWS publishing.

BUILD STATUS 16 automated tests passing	CONTROLLED DEMO Vehicle and face paths verified
PRIMARY PLATFORM Windows + Python 3.11	CLOUD STRATEGY Local first, private AWS second

How to use this guide

Best route for a beginner

Get the included synthetic demo working locally first. Only after every event JSON and evidence crop is visible should you enable AWS. This keeps computer-vision debugging separate from cloud permissions and networking.

Part	Use it for
1-3	Understand what is built and run the first Windows demo.
4-7	Explain each model, the JSON contract, matching and trust score.
8-10	Connect private AWS storage and later container processing.
11-14	Tune real footage, add height/accessories, troubleshoot and present.

The shortest successful path

- 1 Extract the project ZIP into a short path such as `C:\sentinel-camera-ai`.
- 2 Install Python 3.11 and Tesseract 5.
- 3 Run `scripts\setup_windows.ps1`.
- 4 Run `scripts\run_demo.ps1`.
- 5 Open the generated annotated images and event JSON under output.
- 6 Replace synthetic media with short, authorized clips and tune confidence thresholds.
- 7 Configure AWS IAM Identity Center and a private S3 bucket; publish only after local success.

Accuracy promise

No camera AI can be guaranteed never to fail. This build is engineered to fail visibly: uncertain results become Unknown/null, fallbacks are recorded, and cross-camera links remain evidence-ranked candidates.

1. What has been built

The project is a modular Python package and command-line application. It accepts an image or recorded clip and writes a versioned event JSON plus annotated evidence, face crops, plate crops and optional anonymous face vectors.

Capability	Implemented now	What the result means
Face presence	YuNet primary; Haar fallback; boxes, crops, confidence	Detects a face. It does not name or identify a person.
Face comparison	SFace anonymous embedding and cosine similarity	Possible same face with evidence strength; not an identity probability.
Plate	YOLO adapter, LPD-YuNet/contour fallback, PaddleOCR/Tesseract	Separate detection and OCR confidence; normalized SA-style suffix display.
Vehicle	Broad type, dominant colour, movement direction	Make/model remains null unless a reliable future model supplies it.
Appearance	Upper and lower colour; cap/backpack schema fields	Colours work now. Cap/backpack stay null until an attribute model is added.
Repeat pass	Gap-aware in-clip event buckets and cross-event comparison	A return after the cooldown can become a new pass event.
Trust score	Blur, lighting, detections, obstruction and resolution	Quality heuristic out of 100, with readable reasons.
Adaptive edge	NORMAL and HEIGHTENED modes	The system spends more compute only when risk is elevated.
AWS	Private S3 + optional ingestion API publisher	Built and unit-tested without mutating a live AWS account.

Project layout

```

sentinel-camera-ai/
  config/default.yaml      # thresholds, modes, AWS settings
  media/                  # safe synthetic clips and face fixture
  models/                 # official OpenCV models + licences
  schemas/                # JSON Schemas
  scripts/                # Windows setup, demo, model download
  src/sentinel_camera_ai/ # pipeline, detectors, matching, AWS
  tests/                  # unit and end-to-end tests
  demo-output/            # verified example evidence

```

2. Architecture and data flow

1. Media input Image, MP4, AVI, MOV	2. Adaptive mode NORMAL or HEIGHTENED
3. Perception Person/vehicle • face • plate • OCR	4. Evidence quality Trust score • confidence • fallbacks
5. Event builder Crops • annotation • schema v1 JSON	6. Match/publish Vehicle • face • appearance • AWS

Why this structure is reliable

- Adapters, not one giant script: detectors can be replaced without changing the event contract.
- Primary + fallback: optional model failures do not silently destroy the controlled demo.
- One event schema: the team receives the same shape whether input came from an image, clip or future webcam.
- Evidence first: every decision can link back to a crop, annotated frame, confidence and model name.
- Abstention: missing or weak evidence becomes Unknown/null instead of an invented answer.

Runtime sequence

- 1 Open media and sample frames according to the selected mode.
- 2 Optionally gate quiet frames using motion.
- 3 Detect people/vehicles, faces and plate candidates.
- 4 Run heavy OCR only at the configured interval.
- 5 Extract vehicle colour, direction and visible appearance colours.
- 6 Calculate the Camera Trust Score.
- 7 Merge continuous observations; split a return after the cooldown.
- 8 Keep the highest-quality representative frame for each event.
- 9 Write evidence and schema-v1 JSON; optionally publish to AWS.

Distinctive design: Sentinel Mesh decides when to spend compute, rates evidence quality, and records why multi-signal links were made.

3. Windows installation and first run

Prerequisites

Item	Recommended	Why
Python	3.11 x64	Supported by the package and common CV wheels.
Tesseract	5.x on PATH	Offline OCR fallback.
Git	Current stable	Version control and collaboration.
FFmpeg	Current stable	Useful for inspecting/converting clips.
RAM	8 GB minimum; 16 GB preferable	Model loading and video frames.
GPU	Optional	CPU is enough for the controlled demo.

One-command project setup

```
cd C:\sentinel-camera-ai
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\scripts\setup_windows.ps1
```

The setup script creates .venv, installs the package, AWS/YOLO/development extras, downloads official OpenCV Zoo models and runs the dependency doctor.

If Tesseract is not on PATH

```
$env:TESSERACT_CMD =
"C:\Program Files\Tesseract-OCR\tesseract.exe"
```

Run the controlled demo

```
.\scripts\run_demo.ps1
```

Expected result

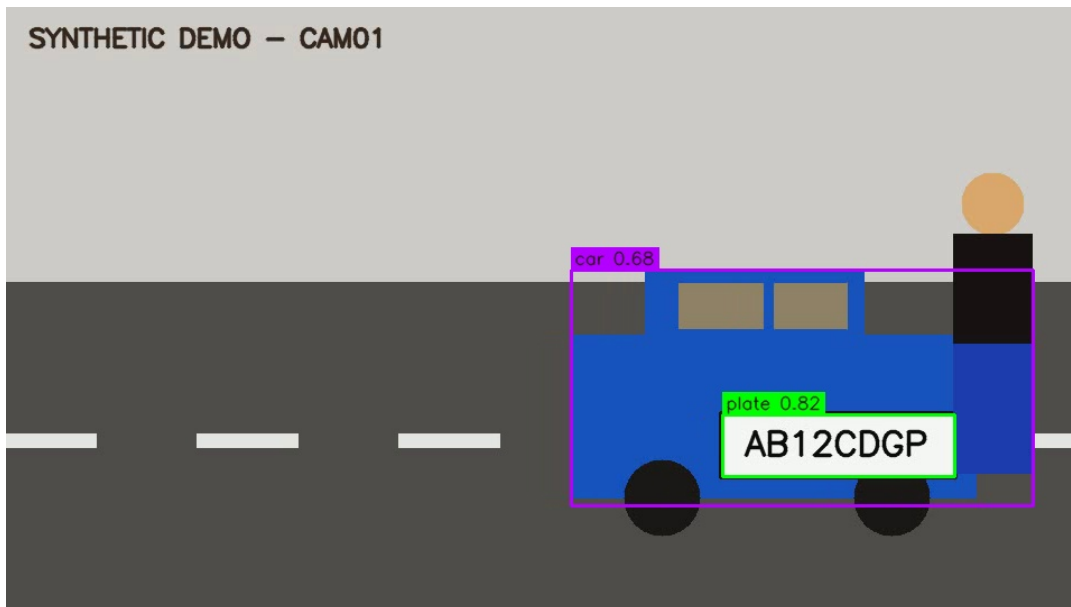
- Two clip events containing plate text AB12CDGP.
- Vehicle evidence Blue / Car / right.
- Camera Trust Scores around the high 70s for selected frames.
- A comparison JSON marking possible same vehicle as true with HIGH evidence.

If setup stops

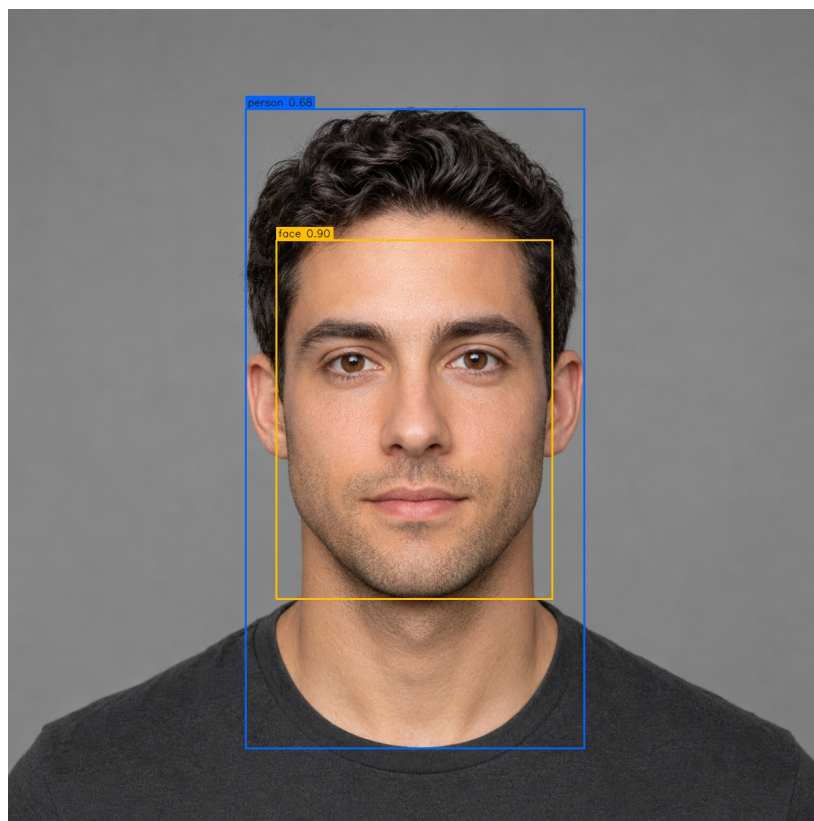
Do not skip the error. Run `python -m sentinel_camera_ai --config config/default.yaml doctor` and use the troubleshooting section later in this guide.

4. What the verified demo produces

These are actual outputs from the packaged build. The media is synthetic, so the team can demonstrate the pipeline without exposing a real face or plate.



Vehicle event: car 0.68, plate candidate 0.82, OCR AB12CDGP, Blue / Car / right, Camera Trust 79/100.



Fictional face fixture: one YuNet face at approximately 0.895 confidence. The crop and SFace embedding were saved; no unreadable plate candidate was retained.

Inspect demo-output/ for both event JSON files, the vehicle comparison, the face comparison and all evidence crops.

5. Detector and OCR strategy

Person and vehicle

- Preferred: Ultralytics-compatible YOLO for person, car, motorcycle, bus and truck.
- Fallback: an OpenCV colour/contour heuristic used only to keep the synthetic integration fixture self-contained.
- Do not judge real CCTV accuracy using the heuristic fallback.

Face

- OpenCV YuNet supplies face boxes, five facial landmarks and detection confidence.
- OpenCV Haar is the last-resort detector when YuNet cannot load.
- OpenCV SFace aligns the detected face and produces an anonymous numeric vector.
- The JSON stores an embedding file reference, not a person's name.

Licence plate

- 1 Use `models/license_plate_detector.pt` when a tested, properly licensed plate-specific YOLO model is available.
- 2 Otherwise combine OpenCV LPD-YuNet proposals with contour proposals.
- 3 Reject impossible plate aspect ratios and oversized candidates.
- 4 Crop the two best candidates and run OCR preprocessing variants.
- 5 Use PaddleOCR when installed; fall back to local Tesseract.
- 6 Normalize to uppercase alphanumeric text and keep detection/OCR confidence separate.

South African limitation

The included synthetic plate demonstrates the required SA-style suffix GP. The bundled LPD-YuNet detector was not trained for South African plates. Before real evaluation, add a geographically suitable plate model and test day/night/angle cases on authorized local examples.

How to add a stronger plate model

```
# Place an Ultralytics-compatible model here:
models\license_plate_detector.pt

# Keep config/default.yaml:
plate:
  backend: auto
  yolo_model: models/license_plate_detector.pt
```

6. Event JSON contract

Every event is validated by Pydantic and can also be checked against `schemas/event-v1.schema.json`. Downstream teammates should integrate against this contract, not detector internals.

JSON area	Important fields	Integration meaning
Identity	<code>schema_version</code> , <code>event_id</code> , <code>camera_id</code>	Stable routing and version handling.
Time/place	timezone-aware timestamp, latitude, longitude	Map and sequence events safely.
Face	<code>present</code> , <code>count</code> , <code>boxes</code> , <code>crops</code> , <code>embedding_ref</code> , <code>confidence</code>	Presence and optional anonymous comparison.
Plate	<code>text</code> , <code>display_text</code> , <code>box</code> , <code>crop</code> , <code>detection/OCR confidence</code>	Searchable vehicle evidence with uncertainty.
Vehicle	<code>colour</code> , <code>type</code> , <code>make_model</code> , <code>direction</code> , <code>box</code>	Broad attributes; no invented make/model.
Appearance	<code>upper/lower colour</code> , <code>cap</code> , <code>backpack</code> , <code>person_box</code>	Visible, non-identity appearance cues.
Quality	<code>camera_trust_score</code> , <code>metrics</code> , <code>reasons</code>	Whether evidence is suitable for linking/review.
Audit	<code>model_versions</code> , <code>metadata</code> , <code>fallback notes</code>	Reproduce and explain the event.

Example core output

```
{
  "event_id": "EVT-4ADFB703C2",
  "camera_id": "CAM01",
  "timestamp": "2026-07-24T21:07:04+02:00",
  "mode": "HEIGHTENED",
  "face": {"present": false, "count": 0},
  "plate": {
    "text": "AB12CDGP",
    "detection_confidence": 0.82,
    "ocr_confidence": 0.72
  },
  "vehicle": {
    "colour": "Blue", "type": "Car", "direction": "right"
  },
  "camera_trust_score": 79
}
```

Validate an event

```
python -m sentinel_camera_ai --config config/default.yaml `
  validate --event output/events/EVT-....json
```

7. Matching, Camera Trust and adaptive modes

Cross-event matching

Decision	Signals	Current rule
Possible same vehicle	Plate 65%, colour 20%, broad type 15%	True at score ≥ 0.76 when both plates exist.
Possible same face	SFace cosine similarity + minimum camera trust	True at similarity ≥ 0.78 and trust ≥ 55 .
Possible same appearance	Upper 45%, lower 35%, optional cap/backpack	True at score ≥ 0.75 with real appearance evidence.

Missing plate or face evidence does not become a false match. The comparison returns NONE/LOW evidence and a warning where appropriate.

Camera Trust Score

Component	Weight	Purpose
Sharpness	25%	Penalizes blur using Laplacian variance.
Lighting	20%	Rewards usable brightness; penalizes very dark/bright frames.
Detection	30%	Aggregates current detector/OCR confidence.
Unobstructed	15%	Penalizes evidence touching/cut by frame boundaries.
Resolution	10%	Rewards sufficient evidence pixels.

Interpretation

78/100 means the selected frame is reasonably usable under the configured heuristic. It is not a probability that the plate or person is correct.

Adaptive processing

Mode	Runs	Use
NORMAL	Sparse frames, basic person/vehicle; face/plate/appearance off	Routine low-risk monitoring.
HEIGHTENED	Dense frames, face, plate/OCR, vehicle and appearance	Alert window or high-risk time.

Continuous detections merge while the observation gap is within `dedupe_cooldown_seconds`. If the object disappears longer than the cooldown and returns, a new pass event can be created.

8. AWS: the recommended rollout

Recommended cloud boundary

Run inference locally for the first demo. Use AWS first for private evidence storage and event sharing. Move video inference into containers only after accuracy and runtime are measured on representative clips.

Stage 1 — private evidence storage

- 1 Create one private S3 bucket in af-south-1 or the team's chosen Region.
- 2 Enable default encryption and block all public access.
- 3 Use IAM Identity Center for developers instead of long-lived access keys.
- 4 Give the processing role only the required object prefixes.
- 5 Configure bucket, region and `aws.enabled`.
- 6 Run a single synthetic event with `--publish-aws`.
- 7 Confirm evidence and event JSON are private before using real authorized media.

Suggested object layout

```
s3://sentinel-mesh-dev/
  incoming/CAM01/<clip-id>.mp4
  evidence/<event-id>/annotated_frame.jpg
  evidence/<event-id>/plate.jpg
  evidence/<event-id>/faces/face_0.jpg
  events/YYYY/MM/DD/<event-id>.json
```

Stage 2 — team API

Set `aws.ingestion_url` only when the backend has an authenticated HTTPS endpoint. Store the API token outside the repository in `SENTINEL_API_TOKEN` or a managed secret.

Stage 3 — managed batch processing

For longer clips, package this project as a container, push to ECR, run it as an ECS/Fargate task and orchestrate it with Step Functions. This avoids forcing large models and long video jobs into a short serverless function.

9. AWS beginner setup

Configure identity

```
aws configure sso
aws sso login --profile sentinel-dev
aws sts get-caller-identity --profile sentinel-dev
```

Configure the application

```
# config/default.yaml
aws:
  enabled: true
  profile: sentinel-dev
  region: af-south-1
  bucket: your-private-sentinel-bucket
  evidence_prefix: evidence
  events_prefix: events
  ingestion_url: ""
```

Publish one test

```
python -m sentinel_camera_ai --config config\default.yaml process `
  --input media\camera_1_clip.mp4 `
  --camera-id CAM01 `
  --mode HEIGHTENED `
  --timestamp "2026-07-24T21:07:00+02:00" `
  --publish-aws
```

Minimum role actions

- s3:PutObject on only the evidence/events prefixes.
- s3:GetObject only if this worker must read incoming clips.
- s3:ListBucket only with a prefix condition if listing is needed.
- KMS permissions only when using a customer-managed encryption key.

Do not do this

Do not paste AWS access keys into YAML, code, screenshots or chat. Do not make the evidence bucket public. Use short-lived SSO credentials locally and an IAM task role in ECS.

Prevent S3 trigger loops

If a future S3 upload triggers processing, separate incoming/ from evidence/, and filter the event to incoming media suffixes. Otherwise the worker's own output upload can trigger another run.

10. Cloud architecture after the local demo

AWS service	Recommended responsibility	Start when
S3	Private incoming clips, evidence and event JSON	Immediately after local success.
IAM Identity Center	Short-lived developer access	Before the first AWS test.
ECR	Store the built camera-AI container	When deploying managed inference.
ECS/Fargate	Run video-processing tasks	When runtime/accuracy is measured.
Step Functions	Start task, wait, retry and record status	When more than one processing step exists.
DynamoDB	Index event metadata and repeat sightings	When the UI needs fast event queries.
API Gateway + backend	Authenticated team ingestion/query API	When frontend integration is ready.
CloudWatch	Logs, failure alarms and processing metrics	With the first deployed worker.
Secrets Manager	Store external API secret if required	Only if an external token is used.

Suggested event sequence

- 1 Authorized clip arrives under S3 incoming.
- 2 A small ingestion component records a job and starts a Step Functions execution.
- 3 Step Functions launches an ECS/Fargate task with the S3 input URI and camera metadata.
- 4 The container downloads the clip, runs this package, and uploads evidence/events.
- 5 The backend indexes event metadata and creates temporary presigned evidence links for authenticated users.
- 6 CloudWatch records duration, failures, low-trust rates and model versions.

Why not start with Lambda video inference?

Short metadata handlers are a good Lambda fit. Full video inference brings large images, models, native libraries and variable runtime. The included container is therefore positioned for Fargate first; Lambda can remain the lightweight trigger or validator.

Cost control

Keep NORMAL mode local, upload event evidence instead of continuous raw video, apply S3 lifecycle rules, cap clip duration, and run Fargate only for queued work.

11. Real-footage accuracy plan

The correct next step is not more features. It is a small labelled validation set that reflects the actual cameras.

Build a consented validation pack

- Day, dusk and night clips from each planned camera angle.
- Near/far vehicles, front/rear plates, blur and partial obstruction.
- Authorized South African plate formats relevant to the demo area.
- Consented same-person and different-person pairs; include caps and changed clothing.
- Negative images containing signs, logos and face-like/plate-like patterns.

Measure the right things

Area	Measure	Do not accept
Plate detection	Precision/recall at IoU ≥ 0.5	Only screenshots of best cases.
Plate OCR	Full-plate exact match and character error rate	Substring success counted as perfect.
Face presence	Precision/recall by face size and lighting	Identity claims from detection confidence.
Repeat match	Same/different pair ROC, false-match rate	One universal threshold without calibration.
Trust score	Correlation with OCR/detection failures	Treating 78 as a probability.
Runtime	Frames/sec, peak RAM, event latency	Testing only on a developer desktop.

Threshold tuning order

- 1 Tune object/face/plate detection confidence for false positives.
- 2 Tune OCR acceptance for full-plate accuracy.
- 3 Tune event stability and cooldown for duplicate vs repeat-pass behavior.
- 4 Tune matching thresholds on same/different pairs.
- 5 Recalibrate Camera Trust components against actual failure rates.
- 6 Freeze config and model checksums before the final demonstration.

Human review rule

Any low-trust, low-evidence or consequential cross-camera match should be presented as a candidate with source frames—not as an automatic identity conclusion.

12. Height, caps and richer points of interest

Current status

Upper/lower clothing colours are implemented. Cap and backpack fields exist but remain null. Absolute height estimation is intentionally not claimed yet.

Height estimation done correctly

Pixel height alone is not real height: a nearby short person can occupy more pixels than a distant tall person. For a fixed camera, calibrate the ground plane and camera geometry.

- 1 Keep the camera fixed; record resolution, mounting height, tilt and lens.
- 2 Mark at least four known points on the walking plane and compute a homography.
- 3 Use a person detector plus pose/keypoints to estimate head and foot points.
- 4 Project the foot point onto the calibrated ground plane.
- 5 Use the vertical camera geometry or a known-height reference to estimate height.
- 6 Reject crouching, sitting, occluded or foot-out-of-frame cases.
- 7 Return a range such as 1.70–1.80 m with calibration confidence, never a false exact value.

Caps, backpacks and memorable appearance

Feature	Practical implementation	Reliability gate
Cap/hat	Add an attribute classifier on the person/head crop or a detector class	Require enough head pixels and confidence.
Backpack/bag	Add an object class and verify overlap with the person box	Avoid matching a nearby person's bag.
Clothing texture	Small colour histogram + coarse pattern tag	Use only with usable lighting/trust.
Person re-ID vector	Optional consented re-ID model on full-body crop	Treat as possible same appearance, not identity.
Vehicle shape	Embedding from the vehicle crop	Combine with plate/colour; never outweigh a conflicting clear plate.

A unique but achievable extension

Create an evidence signature: plate similarity, vehicle embedding, upper/lower colour, cap/backpack, direction, time gap and Camera Trust. Store each component and its availability. A rules layer or calibrated model can then explain which signals caused a repeat-match candidate.

13. Troubleshooting

Symptom	Likely cause	Fix
Python 3.11 not found	Launcher/path issue	Install x64 Python 3.11 and enable Add Python to PATH.
Tesseract unavailable	Not installed or not on PATH	Set TESSERACT_CMD to tesseract.exe.
No events	Unsupported media, strict threshold, no supported object	Run doctor; try synthetic reference; lower only one threshold at a time.
Plate box but no text	Small/blurred crop or OCR unavailable	Inspect plate.jpg; improve capture; install PaddleOCR; keep null if unreadable.
False plate on logo/sign	Contour proposal resembles a plate	Use plate-specific YOLO; require OCR/vehicle context; add negative tests.
Face crop absent	Face too small/turned/occluded	Use HEIGHTENED, check YuNet model, test frontal authorized fixture.
YOLO warning	Optional ultralytics package/model missing	Run setup extra and provide authorized weights; fallback still supports demo.
AWS AccessDenied	Wrong profile/role/prefix	Run sts get-caller-identity; inspect role and exact S3 resource ARN.
AWS publish loops	Output prefix triggers input rule	Filter only incoming prefix and video suffix.
Slow processing	Too many sampled/OCR frames	Increase frame_stride or heavy_frame_interval; shorten clips.

Diagnostic commands

```
python -m sentinel_camera_ai --config config\default.yaml doctor
python -m pytest -q
python -m sentinel_camera_ai --help

# Verify model checksums on Linux/macOS:
cd models
sha256sum -c checksums.sha256
```

Safe debugging order

- 1 Reproduce with the packaged synthetic fixture.
- 2 Read the event's model_versions and model_notes.
- 3 Inspect best_frame.jpg, annotated_frame.jpg and crops.
- 4 Change one detector/config setting.
- 5 Run tests and compare the same media again.
- 6 Only then check AWS publishing.

14. How to explain the idea to the team

One-sentence pitch

My Camera AI turns short camera clips into explainable, quality-scored events and links possible repeats across cameras without uploading every frame or pretending uncertain evidence is certain.

A 90-second explanation

“I built a local-first camera intelligence pipeline. NORMAL mode uses less compute for basic vehicle/person detection; HEIGHTENED mode adds face presence, plate OCR and appearance. Each result becomes versioned JSON with evidence, confidence, model names and a quality score for blur, lighting, obstruction and resolution. Cross-camera comparison combines plate, vehicle, anonymous face and appearance evidence, then reports a possible match with reasons. The demo is local and synthetic; AWS comes next for private storage and, only if needed, managed Fargate processing.”

Five-minute live demo order

- 1 Run doctor and show that core dependencies/models are available.
- 2 Run scripts\run_demo.ps1.
- 3 Show the annotated vehicle frames, plate crops and one event JSON.
- 4 Open comparison.json and show the plate/colour/type reasons.
- 5 Show the synthetic face event, then switch to NORMAL mode.
- 6 Finish with the private S3/Fargate rollout—not an untested live deployment.

Questions teammates may ask

Question	Strong answer
Is face confidence identity confidence?	No. Detection confidence only means a face-like region was found. Similarity is separate.
Why not upload all video?	Cost, bandwidth and privacy. Adaptive local processing uploads compact evidence/events.
What makes this unique?	Adaptive compute + quality score + multi-signal, explainable repeat linking with abstention.
Will it always work?	No CV system will. We expose uncertainty and validate on representative authorized data.

15. Verification and handoff checklist

Verified in the packaged build

- 16 automated tests pass.
- Source compiles successfully.
- Mock SA-style plate AB12CDGP is read in both synthetic clips.
- Vehicle repeat match returns true, score 1.0, HIGH evidence.
- Fictional face detection returns one face around 0.895 confidence.
- Face crop and SFace anonymous embedding are saved.
- Synthetic face comparison returns true, score 1.0, HIGH evidence.
- NORMAL/HEIGHTENED behavior and JSON validation are tested.
- OpenCV model checksums and upstream licence notices are included.

Not falsely claimed

- Docker files are included but Docker was unavailable in the build environment.
- The AWS adapter is unit-tested with fake clients; no live account was changed.
- Real South African CCTV accuracy requires a local validation set and plate-specific model.
- Caps/backpacks and calibrated absolute height are documented extensions, not current outputs.

Before the team demo

Check	Owner	Done
Run setup and demo on the actual Windows laptop	Camera AI	☐
Record Python/Tesseract/GPU versions	Camera AI	☐
Freeze model files and checksums	Camera AI	☐
Test projector-visible annotated images	Presenter	☐
Agree JSON fields with backend teammate	Camera AI + backend	☐
Use only synthetic/authorized media	Whole team	☐
Prepare local fallback if Wi-Fi/AWS fails	Whole team	☐

Definition of done

The demo is ready when a clean Windows machine can run the two scripts, produce valid events, show evidence, compare repeats and explain every score without needing live AWS or unconsented data.

Official references

Use these primary sources when updating dependencies or deployment choices. They may change after this guide's build date.

- [OpenCV Zoo](#) — official model repository
- [Ultralytics](#) — official documentation
- [PaddleOCR](#) — official quick start
- [AWS IAM best practices](#)
- [AWS CLI IAM Identity Center configuration](#)
- [AWS Lambda with S3 events](#)
- [AWS Lambda container images](#)
- [AWS Step Functions integration with ECS/Fargate](#)

Files to read next

File	Purpose
README.md	Fast project usage and architecture.
BUILD_REPORT.md	Exact verified and unverified claims.
config/default.yaml	Thresholds, modes, paths and AWS settings.
THIRD_PARTY_MODELS.md	Model provenance and licence cautions.
schemas/event-v1.schema.json	Backend integration contract.
demo-output/	Evidence and comparison examples.

Final recommendation: demonstrate the verified local pipeline first, collect a small authorized South African validation set second, and enable private AWS publishing third. That order gives the team a working system, measurable accuracy and a cloud path without combining every risk at once.